

Assignment 1a: SVMs in Practice

1a.1 Margin Maximization in 1D (20 points)

Consider the following simple 1D data set. The first component corresponds to the input x , and the second component corresponds to the target y :

$$\{(-3, +1), (1, -1)\}$$

Determine the exact solution (w, b) of the margin maximization problem.

Show sufficient detail by performing the following steps:

1. Formulate the primal problem.
2. Determine the Lagrange function and the dual problem.
3. Solve the dual problem.
4. Determine w and b from the solution(s) of the dual problem.

1a.2 SVM Classification on the Wine Quality Data Set (40 points)

Consider the Wine Quality Data Set from the UCI Machine Learning Repository:

[winequality-white.csv](#)

Use the white wine data set for a classification task. The last column, `quality`, is the original label column, and the other columns are the input features. Convert the original numeric wine quality score into three classes using the following score ranges:

- `low` : quality scores from 3 to 4
- `medium` : quality scores from 5 to 7
- `high` : quality scores from 8 to 9

Split the data into 70% training samples and 30% test samples. Make sure that the input columns are properly normalized and that preprocessing is fit only on the training data.

Use the following machine learning methods along with hyperparameter optimization:

- SVM with linear kernel and several choices of C .
- SVM with RBF kernel and several choices of C and γ .
- One other classification algorithm of your choice, for example random forest or gradient-boosted trees.

Perform hyperparameter optimization on the training set with respect to a suitable classification score. Then evaluate the best model for each method once on the test set. Carefully structure your analysis and document the results. Discuss the results in the notebook: What do you observe? How do you interpret the results?

Assignment 1b: FCNN Basics

1b.1 Architecture Search (40 points)

Train fully connected neural networks for recognizing handwritten digits using the [KMnist Handwritten Dataset](#). Try to build and find a suitable architecture with at least 3 hidden layers. Adapt this architecture by hand by using all the layers and tricks we have talked about, such as dropout, batch normalization, and skip connections. Feel free to research common approaches to structuring FCNNs. Train at least a few models to get a feel for what works.

Using the intuitions you built there, create python functions to do a simple hyperparameter search over a defined search space. Use this random search over at least 30 configurations. Think about callbacks, such as Early Stopping and the number of epochs to use.

Do a random search over a defined space for at least the following hyperparams as combinations:

- Learning rate
- Activation function
- Number of hidden layers and number of neurons in each hidden layer (vary)
- Dropout
- Optimizer

Hints:

- For continuous variables such as learning rate, use a random variable in a specified range. Check out the Keras documentation.
- For code, create a subfunction that takes hyperparameter ranges and returns a list of dictionaries for the defined combinations to search over.

Deliverables: A notebook or script+markdown demonstrating your implementation with clear explanations and a summary of your findings. I expect an analysis and reasoning over findings, not just raw evaluation results.

Note any use of an LLM in detail please.

Submission

Submission via Moodle.

Total Points: 100